Лабораторная работа № 3

Тема: Простое интеграционное тестирование REST API с Jest и Supertest

Цель: Проверить ответы действующего Express API без изменения базы данных и без перестройки проекта.

Дисциплина: Node.js и Express.js. РО 4.5

Что такое интеграционное тестирование?

Интеграционный тест проверяет совместную работу компонентов. В этой работе HTTP-запрос проходит от Supertest через Express и обработчик маршрута до PostgreSQL, а обратно возвращается ответ API. Мы проверяем именно результат HTTP-запроса, а не одну отдельную функцию.

Supertest — библиотека, которая отправляет HTTP-запросы из теста и позволяет проверить ответ сервера. Jest запускает проверки и показывает PASS или FAIL.

> **Важно**
>
> Эта упрощённая работа только ЧИТАЕТ данные. Она не создаёт, не изменяет и не удаляет записи. Для серьёзных тестов, которые меняют данные, позже понадобится отдельная тестовая база данных. Нельзя проверять удаление или очистку на рабочей базе.

1. Проверьте, что сервер работает

В первом терминале откройте каталог проекта и запустите существующий Express-сервер. Не закрывайте первый терминал до окончания работы.

```
npm start
```

В браузере откройте http://localhost:3000/api/products. Должен появиться JSON-ответ со списком товаров. Проверьте также http://localhost:3000/api/categories. Если адреса вашего API отличаются, укажите реальные маршруты в тесте ниже.

> **Важно**
>
> В этой работе Supertest подключается к уже работающему серверу http://localhost:3000. Поэтому разделять app.js и server.js и изменять существующий backend НЕ нужно.

2. Установите Supertest

Откройте ВТОРОЙ терминал PowerShell в том же каталоге проекта. Выполните:

```
npm install --save-dev supertest
```

Jest из ЛР-2 уже установлен. Дополнительно переустанавливать его не нужно.

3. Обновите настройки Jest

Откройте \jest.config.js. ЗАМЕНИТЕ содержимое файла целиком следующим кодом:

```
export default {
  testEnvironment: "node",
  transform: {},
  clearMocks: true,
  testMatch: [
    "**/tests/unit/orderMath.test.js",
    "**/tests/integration/products.api.test.js"
  ]
};
```

Теперь Jest запускает тесты из упрощённой ЛР-2 и новый файл ЛР-3. Старый tests/orderMath.test.js (из ЛР-1) и тесты Product Service не попадут в этот набор.

4. Создайте файл интеграционных тестов

Создайте папку \tests\integration (если её ещё нет), затем файл products.api.test.js. Вставьте в файл ВЕСЬ код ниже. Больше ничего добавлять в него пока не требуется.

```
import { describe, expect, test } from "@jest/globals";
import request from "supertest";

// Адрес сервера, запущенного командой npm start
const api = request("http://localhost:3000");

describe("Internet Shop API", () => {
  // 1. Получаем список товаров
  test("GET /api/products возвращает 200", async () => {
    const response = await api.get("/api/products");

    expect(response.status).toBe(200);
    expect(response.headers["content-type"]).toMatch(/json/);
  });

  // 2. Получаем список категорий
  test("GET /api/categories возвращает 200", async () => {
    const response = await api.get("/api/categories");

    expect(response.status).toBe(200);
  });

  // 3. Проверяем несуществующий маршрут
  test("неизвестный маршрут возвращает 404", async () => {
    const response = await api.get("/api/no-such-route-lab3");

    expect(response.status).toBe(404);
  });
});
```

5. Разберитесь в трёх проверках

Выражение	Что означает
request("http://localhost:3000")	Отправлять запросы работающему серверу
api.get("/api/products")	Выполнить HTTP-запрос GET /api/products
response.status	Полученный HTTP-статус

Выражение	Что означает
expect(...).toBe(200)	Проверить, что сервер ответил 200 OK
toMatch(/json/)	Проверить, что Content-Type содержит json
expect(...).toBe(404)	Проверить, что неизвестный маршрут не найден

Для первого теста не важно, сколько товаров есть в базе: мы проверяем наличие успешного JSON-ответа, а не заранее заданное количество записей.

6. Запустите только тесты ЛР-3

Первый терминал с сервером должен продолжать работать. Во втором терминале выполните:

```
npm test -- tests/integration/products.api.test.js
```

При работающем сервере и указанных маршрутах ожидаются три успешно выполненных теста. Jest может показывать предупреждение ExperimentalWarning о VM Modules; оно не означает провал проверки.

```
PASS tests/integration/products.api.test.js
Tests: 3 passed, 3 total
```

Если возникло ECONNREFUSED — Express не запущен или работает не на порту 3000. Если получен 404 вместо 200 — проверьте адрес соответствующего маршрута в приложении.

7. Убедитесь, что Jest действительно находит ошибку

В файле tests/integration/products.api.test.js временно замените в первом тесте expect(response.status).toBe(200) на expect(response.status).toBe(201). Снова выполните команду из шага 6. Jest должен показать FAIL: ожидается 201, получено 200.

Сразу верните 200 и повторите запуск. Результат снова должен быть PASS.

8. Самостоятельное задание

В том же файле tests/integration/products.api.test.js ВНУТРИ блока describe(), ПЕРЕД последней строкой }); добавьте четвёртый тест. Он должен отправлять GET /api/products и проверять только HTTP-статус 200. Возьмите за образец второй тест для категорий, изменив адрес и название.

Запустите npm test -- tests/integration/products.api.test.js. После добавления ожидается четыре успешно выполненных теста. Самостоятельный тест не должен удалять или менять данные.

9. Что сдавать

1. Файл jest.config.js с двумя явно указанными путями тестов.
2. Файл tests/integration/products.api.test.js с четырьмя проверками (три основных и одна самостоятельная).
3. Скриншот терминала с успешным запуском Jest.

Контрольные вопросы

4. Чем интеграционный тест отличается от модульного?
5. Для чего нужен Supertest?
6. Что означает HTTP 200?
7. Почему на неизвестный маршрут ожидается HTTP 404?
8. Почему в этой работе не изменяем содержимое базы данных?

Критерии оценивания

Критерий	Баллы
Установлен Supertest, работает Express	20
Jest запускает только нужные файлы	20
Проверены GET /api/products и GET /api/categories	25
Проверены JSON-заголовок и статус 404	20
Добавлен самостоятельный тест	15